Программная инженерия

Лекция 4: Качество кода. Чистый код и рефакторинг

Денисов Артём Владимирович

Уральский Федеральный Университет

2026

Код как профессиональная среда

Ключевые моменты

- Кодовая база — наша “комната”
- Малые решения в начале формируют хорошую/плохую в будущем
- Порядок или хаос — результат дисциплины

Какой вариант предпочтительнее?

Аккуратная система

- Понятные модули
- Предсказуемое поведение
- Тесты дают уверенность

Спагетти код и хаос

- Странные зависимости
- Copy–paste
- Сложность что-либо менять

Качество кода и скорость разработки

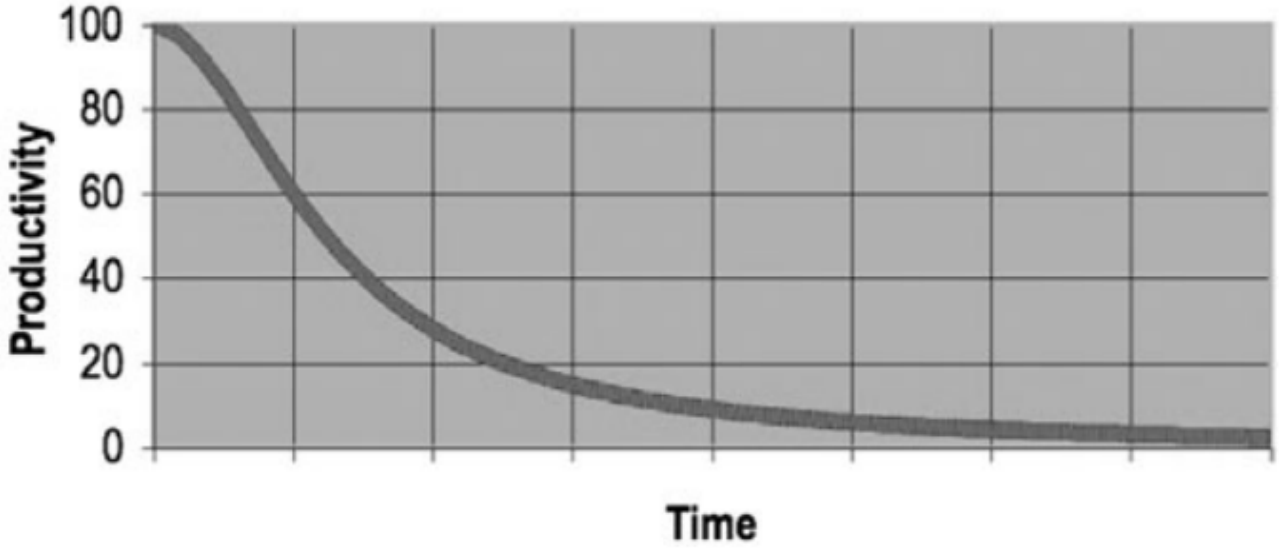
Что происходит при низком качестве?

- Каждое изменение рискованно
- Фича = “минное поле”
- Сложность выходит из-под контроля

Что даёт чистый код?

- Контролируемая сложность
- Быстрое добавление поведения
- Стабильная долгосрочная скорость

Рост стоимости изменений



- Низкое качество уменьшает продуктивность
- «Ржавчина» накапливается постепенно

Проблема в крупных компаниях

Ситуация

- Огромные кодовые базы
- Тысячи разработчиков
- Долгий жизненный цикл систем

Как поддерживать такую большую систему?

Чистому коду можно научиться

«Any fool can write code that a computer can understand. Good programmers write code that humans can understand.»

— Martin Fowler

Почему это важно?

- Код читают чаще, чем пишут
- Мы пишем для людей, не только машин

Изменения — норма

Реальность проектов

- Требования постоянно меняются
- Новые интеграции и регуляции
- Долгоживущие системы

Хороший дизайн

- Облегчает изменения
- Поддерживает инкрементальную разработку
- Уменьшает «большие переписывания»

Что делает ИИ?

ИИ ускоряет набор кода, не качество

- Работает быстро и результативно
- Закрывает 80–90% задач
- Ускоряет написание фич

Что остаётся человеку

- Архитектурные решения
- Безопасность
- Соответствие практикам командной разработки

Итоги: Просто рабочий код - недостаточно?

Хороший код

- Качество снижает стоимость изменений
- Улучшает надёжность и коммуникацию
- Поддерживает скорость и гибкость
- Контролирует действия ИИ агентов

Этот код чистый?

```
def process(f):
    t=0
    for line in open(f):
        p=line.split(',')
        if len(p)==4:
            q,pr,d=int(p[1]),float(p[2]),float(p[3])
            amt=q*pr*(1-d/100)*1.08
            print(f"{p[0]}: ${amt:.2f}")
            t+=amt
    print(f"Total: ${t:.2f}")

process("orders.csv")
```

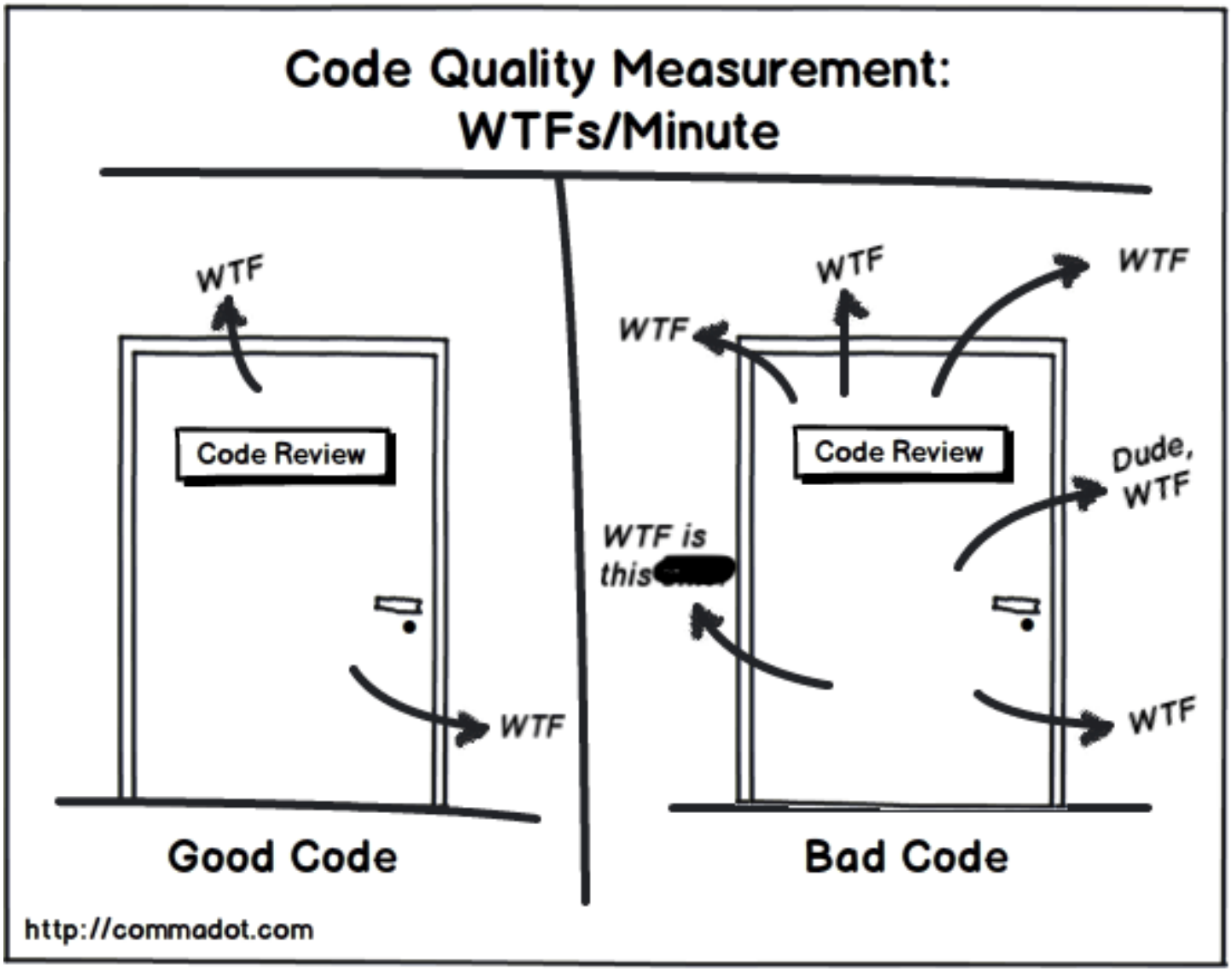
Этот код чистый?

```python
def calc_total(qty, price, disc):
    subtotal = qty * price * (1 - disc/100)
    return subtotal * 1.08

def process_orders(filename):
    total = 0
    with open(filename) as f:
        for line in f:
            try:
                uid, qty, price, disc = line.strip().split(',')
                order_total = calc_total(int(qty), float(price), float(disc))
                print(f"Order {uid}: ${order_total:.2f}")
                total += order_total
            except ValueError:
                continue
    print(f"Total: ${total:.2f}")
```

Этот код чистый?

```
TAX_RATE = 0.08

@dataclass
class Order:
    user_id: str
    quantity: int
    price: float
    discount: float

    @property
    def total(self) -> float:
        return self.quantity * self.price * (1 - self.discount/100) * (1 +
TAX_RATE)

def parse_orders(filename: str) -> List[Order]:
    orders = []
    with open(filename) as f:
        for line in f:
            try:
                uid, qty, price, disc = line.strip().split(',')
                orders.append(Order(uid, int(qty), float(price), float(disc)))
            except ValueError:
                continue
    return orders

orders = parse_orders("orders.csv")
for order in orders:
    print(f"Order {order.user_id}: ${order.total:.2f}")
print(f"Total: ${sum(o.total for o in orders):.2f}")
```

Что такое чистый код?



Мнения экспертов

«Clean code is simple and direct. Clean code reads like well-written prose. Clean code never obscures the designer's intent but rather is full of crisp abstractions and straightforward lines of control.»

— Grady Booch

Мнения экспертов

«Clean code can be read, and enhanced by a developer other than its original author. It has unit and acceptance tests. It has meaningful names.»

— Dave Thomas

Мнения экспертов

«You know you are working on clean code when each routine you read turns out to be pretty much what you expected.»

— Ward Cunningham

Признаки чистого кода

Чистый код

- Понятный и читаемый
- Поддаётся сопровождению и изменению
- Простой и аккуратный
- Последовательный и стандартизованный
- Не дублируется (DRY)
- …

Грязный код порождает технический долг

Что это?

Технический долг — это накопившаяся «задолженность» в виде упрощений, костылей и неаккуратных решений в коде, за которую позже приходится платить ростом стоимости изменений и сопровождения.

Технический долг и качество кода

Ключевые характеристики

- снижает качество кода
- усложняет понимание и изменение системы
- увеличивает стоимость сопровождения

Намеренный vs случайный технический долг

Намеренный долг

- сознательный компромисс ради скорости
- используется для MVP или дедлайна
- планируется последующий рефакторинг

Случайный долг

- возникает из-за незнания или спешки
- плохие практики или слабый процесс
- часто долго остаётся незаметным

Открытый vs скрытый технический долг

Открытый долг

- известен всей команде
- задокументирован в backlog
- запланирован рефакторинг

Скрытый долг

- никто не осознаёт проблему
- слабая документация и тесты
- проявляется через инциденты

Как накапливается технический долг

Основные причины

- выбор быстрее вместо лучше
- давление дедлайнов
- отсутствие рефакторинга и тестов

Технический долг - не всегда плохо

Открытый технический долг полезен

- Технический долг есть во всех проектах
- Чем больше из этого долга открыто, задокументированно - тем лучше
- Виден “фронт работ”

Скрытый технический долг - плохо!

Code smells

Определение

Code smells — это распознаваемые признаки потенциальных проблем в дизайне или реализации, которые указывают, что код, вероятно, нуждается в рефакторинге, хотя формально он может работать правильно.

Code Smells: обзор

Общие проблемы

- дублирование кода
- магические числа
- проблемные комментарии
- непонятные имена
- мёртвый код

Функции и структура

- длинные функции
- длинный список параметров
- неверный уровень абстракции
- непоследовательность
- функциональная зависть
- большие классы

Дублирование кода

```python
class OrderProcessor:
    def calculate_tax_eu(self, order):
        subtotal = sum(item.price * item.quantity for item in order.items)
        tax = subtotal * 0.21
        discount = 5 if subtotal > 100 else 0
        return subtotal + tax - discount

    def calculate_tax_us(self, order):
        subtotal = sum(item.price * item.quantity for item in order.items)
        tax = subtotal * 0.08
        discount = 5 if subtotal > 100 else 0  # дублирование логики расчета
        return subtotal + tax - discount
```

Одна и та же логика повторяется в нескольких местах, из-за чего любое изменение требует правок в разных частях системы

Дублирование кода. Решение

```python
class OrderProcessor:
    def _calculate_subtotal(self, order):
        return sum(item.price * item.quantity for item in order.items)

    def calculate_tax(self, order, tax_rate):
        subtotal = self._calculate_subtotal(order)
        tax = subtotal * tax_rate
        discount = 5 if subtotal > 100 else 0
        return subtotal + tax - discount

    def calculate_tax_eu(self, order): return self.calculate_tax(order, 0.21)
    def calculate_tax_us(self, order): return self.calculate_tax(order, 0.08)
```

Повторяющийся код - упущенная возможность для абстракции

Магические числа (Magic Numbers)

```
class ReportGenerator:
    def generate_monthly_report(self, sales_data):
        avg_sales = sum(sales_data) / len(sales_data)
        high_performers = [sale for sale in sales_data if sale > 15000]  # порог
успеха?
        margin = sum(sale * 0.35 for sale in high_performers)  # маржа?
        return {"avg": avg_sales, "high": len(high_performers), "margin": margin}
```

Числа используются напрямую в коде без объяснения их смысла

Магические числа. Решение

```
class ReportGenerator:
    HIGH_PERFORMANCE_THRESHOLD = 15000
    PROFIT_MARGIN = 0.35

    def generate_monthly_report(self, sales_data):
        avg_sales = sum(sales_data) / len(sales_data)
        high_performers = [sale for sale in sales_data if sale >
self.HIGH_PERFORMANCE_THRESHOLD]
        margin = sum(sale * self.PROFIT_MARGIN for sale in high_performers)
        return {"avg": avg_sales, "high": len(high_performers), "margin": margin}
```

Заменить магические числа на константы

Проблемные комментарии

```python
class OrderProcessor:
    def process_payment(self, order):
        total = order.calculate_total()

        # is this a credit card payment or bank transfer?
        if order.payment_type == "card":
            self.charge_card(total, order.customer.card)
        # handles PayPal payments differently
        elif order.payment_type == "paypal":
            self.process_paypal(total, order.customer.email)
        else:
            # default to bank transfer
            self.initiate_bank_transfer(total)
```

Комментарии внутри кода - признак грязного кода

Закоментированный код - табу!

Проблемные комментарии. Решение

```python
class OrderProcessor:
    def process_payment(self, order):
        total = order.calculate_total()

        if order.is_credit_card_payment():
            self.charge_card(total, order.customer.card)
        elif order.is_paypal_payment():
            self.process_paypal(total, order.customer.email)
        else:
            self.initiate_bank_transfer(total)
```

Вместо комментариев - правильно названные сущности

Комментарии в коде - антипаттерн

«The proper use of comments is to compensate for our failure to express ourself in code»

— Robert Martin

- Код эволюционирует, комментарии нет.
- Устаревший комментарий - худший враг программиста.

Непонятные имена сущностей

```
class InventoryManager:
    def _proc(self, itms, wh):
        st = sum(i.q for i in itms if i.w == wh)
        if st < 10:
            self._ord(i.s for i in itms)
        return st
```

Имена переменных не объясняют код

Непонятные имена сущностей. Решение

```
class InventoryManager:
    MIN_STOCK_LEVEL = 10

    def get_stock_level(self, items, warehouse):
        stock = sum(item.quantity for item in items if item.warehouse == warehouse)
        if stock < self.MIN_STOCK_LEVEL:
            self.order_more(items)
        return stock
```

Используйте выразительные имена, которые точно описывают назначение переменной, функции или класса. Длинные имена - не всегда плохо

Мёртвый код (Dead Code)

```python
class PaymentGateway:
    def process_payment(self, amount, card):
        # Поддержка PayPal удалена, но код остался
        # if gateway == "paypal":
        #     return self._paypal_charge(amount)

        result = self.stripe.charge(amount, card)
        self._log_transaction(result)

        old_feature = "legacy"  # не используется
        return result
```

Фрагменты кода остаются в системе, хотя больше никогда не выполняются

Мёртвый код. Решение

```
class PaymentGateway:
    def process_payment(self, amount, card):
        result = self.stripe.charge(amount, card)
        self._log_transaction(result)
        return result
```

Удалите неиспользуемый код. При необходимости он есть в git

Длинная функция (Long Method)

```python
def generate_invoice_pdf(customer, orders):
    pdf.add_text("Invoice", size=20)
    pdf.add_text(f"Customer: {customer.name}", size=12)

    subtotal = 0
    for order in orders:
        subtotal += order.total
        pdf.add_text(f"Order {order.id}: ${order.total}")

    tax = subtotal * 0.21
    total = subtotal + tax

    pdf.add_text(f"Subtotal: ${subtotal}")
    pdf.add_text(f"Tax: ${tax}")
    pdf.add_text(f"Total: ${total}")

    pdf.save("invoice.pdf")
    send_email(customer.email, "Invoice attached", "invoice.pdf")
    log_invoice(customer.id, total)
```

Функция выполняет слишком много задач, из-за чего её сложно читать, тестировать и изменять

Длинная функция. Решение

```
def generate_invoice_pdf(customer, orders):
    pdf = PDF()
    pdf.add_banner()
    pdf.add_customer_info(customer)
    pdf.add_orders(orders)
    pdf.add_totals(calculate_totals(orders))
    pdf.save_and_email(customer.email)
```

Разбейте функцию на несколько небольших методов, каждый из которых выполняет одну чёткую задачу

Длинный список параметров

```
def create_user_report(user_id, start_date, end_date, include_details,
                       filter_by_status, group_by_region, currency,
                       export_format, template_id, watermark_text):
    query = f"SELECT * FROM reports WHERE user_id={user_id} AND date BETWEEN
'{start_date}' AND '{end_date}'"
    # ... сложная логика с 10 параметрами
```

Метод принимает слишком много аргументов, что ухудшает читаемость и повышает вероятность ошибок при вызове

Длинный список параметров. Решение

```python
@dataclass
class ReportConfig:
    user_id: int
    start_date: str
    end_date: str
    include_details: bool
    filter_by_status: str = None

def create_user_report(config: ReportConfig, advanced_options: dict):
    query = build_query(config)
    return generate_pdf(query, **advanced_options)
```

Сгруппируйте связанные параметры в объект

Неверный уровень абстракции

```
def process_salary_payment(employee):
    if employee.is_manager():
        bonus = employee.base_salary * 1.2
    else:
        bonus = 0

    conn = psycopg2.connect("host=localhost dbname=payroll")
    cur = conn.cursor()
    cur.execute("INSERT INTO payments (emp_id, amount, bonus) VALUES (%s, %s, %s)",
                (employee.id, employee.base_salary + bonus, bonus))
    conn.commit()
```

В одном месте смешиваются высокоуровневая бизнес-логика и низкоуровневые детали реализации

Неверный уровень абстракции. Решение

```
class PayrollService:
    def process_salary_payment(self, employee):
        payment_amount = self._calculate_salary(employee)
        self.payment_repository.save_payment(employee.id, payment_amount)

    def _calculate_salary(self, employee):
        base = employee.base_salary
        bonus = base * 1.2 if employee.is_manager() else 0
        return base + bonus
```

Разделите высокоуровневые операции и детали реализации по разным методам и классам соответствующего уровня

Непоследовательность (Inconsistency)

```
def parse_date(date_str): return datetime.strptime(date_str, "%d.%m.%Y")
def format_currency_eu(amount): return f"{amount:,.2f}".replace(",", "
").replace(".", ",")
def format_currency_us(amount): return "{:.2f}".format(amount)
def validate_email(email): return bool(re.match(r"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.
[a-zA-Z0-9-.]+$", email))
```

Одинаковые задачи реализованы разными способами

Непоследовательность. Решение

```
class Formatter:
    @staticmethod
    def format_date(date_obj): return date_obj.strftime("%d.%m.%Y")
    @staticmethod
    def parse_date(date_str): return datetime.strptime(date_str, "%d.%m.%Y")
    @staticmethod
    def format_currency(amount, locale="eu"):
        return localeconv()['decimal_point'].join(f"{amount:,.2f}".split(','))
    @staticmethod
    def validate_email(email):
        return bool(re.match(r"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$",
email))
```

Стандартизируйте подход к решению однотипных задач — используйте общие паттерны и соглашения

Функциональная зависть (Feature Envy)

```python
class Order:
    def __init__(self, customer):
        self.customer = customer
        self.items = []

    def print_report(self):
        print(f"Customer: {self.customer.name} ({self.customer.address.street})")
        total = sum(item.price * item.quantity for item in self.items)
        print(f"Total: ${total}")
        if self.customer.is_vip():
            print("VIP discount applied")
```

Метод слишком активно использует данные другого класса и логически должен находиться внутри этого класса

Функциональная зависть. Решение

```python
class Order:
    def print_report(self):
        print(f"Order total: ${self.total}")
        self.customer.print_details()

class Customer:
    def print_details(self):
        print(f"Customer: {self.name} ({self.address.street})")
        if self.is_vip():
            print("VIP discount applied")
```

Переместите метод в тот класс, данные которого он преимущественно использует

Большой класс (Large Class)

```python
class ShoppingCart:
    def login_user(self, creds): pass
    def validate_session(self): pass
    def add_item(self, product): pass
    def remove_item(self, id): pass
    def process_payment(self, card): pass
    def send_confirmation(self): pass
    # 26 методов...
```

Класс содержит слишком много обязанностей

Большой класс. Решение

```python
class ShoppingCart:
    def add_item(self, product): pass
    def remove_item(self, id): pass
    def calculate_total(self): pass

class AuthService:
    def login_user(self, creds): pass
    def validate_session(self): pass

class PaymentService:
    def process_payment(self, cart): pass
```

Разделите класс на несколько более мелких, каждый из которых отвечает за одну область функциональности

Базовые правила рефакторинга

- Правило бойскаута
- Прагматизм лучше, чем перфекционизм
- Малые шаги с проверкой тестами

Правило бойскаута

«Always leave the code a little cleaner (better) than you found it»

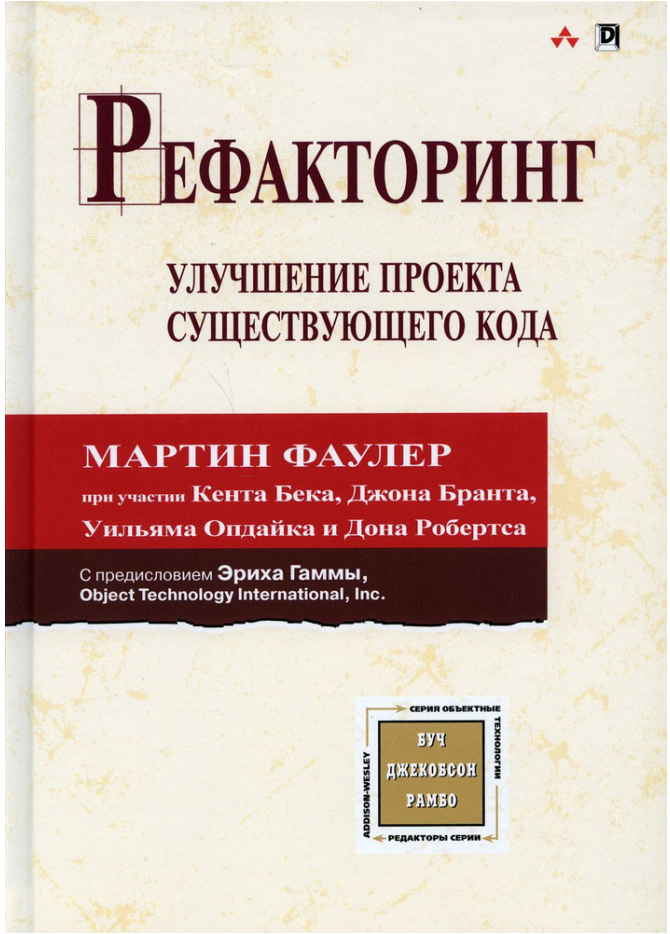
- Улучшай код при каждой небольшой возможности
- Небольшие улучшения накапливаются
- Система постепенно становится лучше

Баланс: не стремиться к идеалу сразу

Прагматичный подход:

- Рефакторинг — непрерывный процесс
- Улучшай код постепенно
- Учитывай сроки и приоритеты задач

Литература



Refactoring. Martin Fowler, Kent Beck, 1999

Clean Code. Robert Martin, 2008

Clean code - Uncle Bob - all lessons

Clean code - Uncle Bob

https://www.youtube.com/watch?v=7EmboKQH8lM&list=PLmmYSbUCWJ4x1GO839azG_BBw8rkh-zOj

Насколько «чистый» код пишет ИИ?

Общая картина

- «Сырой» LLM-код часто содержит дефекты и запахи
- Итоговое качество сильно зависит от ревью и рефакторинга

Ключевой тезис

- ИИ пишет рабочий, но не автоматически «чистый» код

Исследование GitClear, 2020–2023

Ключевой тезис

Массовое использование GitHub Copilot и других AI-инструментов оказывает «нисходящее давление» на качество кода

Методология исследования

- **Масштаб:** 153 млн изменённых строк кода
- **Период:** 48 месяцев (2020–2023)
- **Источники:** GitHub, GitLab, Bitbucket
- **Типы репозиториев:** коммерческие и open-source

Результат 1: Рост «churn»

Определение

Churn — код, удалённый или сильно изменённый менее чем через 2 недели после написания

- Заметный рост с 2022 года (начало массового использования Copilot)

Интерпретация

Разработчики чаще пишут код, который быстро оказывается ненужным или требует переработки

Результат 2: Нарушение принципа DRY

Увеличилась доля:

- ↑ Добавление нового кода
- ↑ Копирование/вставка

Снизилась доля:

- ↓ Перемещение кода
- ↓ Рефакторинг

Результат 3: Сокращение жизненного цикла кода

- **Возросла** доля кода, изменяемого в первые 2 недели
- **Уменьшилась** доля кода, живущего дольше месяца–года

Что это означает

Более частое появление:

- «Лишних» изменений
- Плохо продуманных решений
- Недостаточно протестированного кода

Выводы

1. AI-автодополнения ускоряют написание кода, но снижают его качество
2. Растёт объём «одноразового» кода с коротким жизненным циклом
3. Нарушаются классические принципы разработки (DRY, рефакторинг)

Типичные проблемы ИИ-кода

Что происходит на практике

- Избыточный и многословный код
- Дублирование фрагментов и паттернов
- Смешение уровней абстракций в одной функции
- Длинные методы с разной логикой внутри

Роль разработчика: ревью ИИ-кода

Что делаем всегда

- Рассматриваем вывод ИИ как черновик для ревью
- Проверяем структуру, читаемость, тестируемость

Как просить ИИ о рефакторинге

Принцип конкретности

- Просим менять конкретные участки, а не “всё”
- Формулируем цель: читаемость, модульность, удаление дубликатов
- Ограничиваем область: один refactoring-заход — одна проблема

Куда смотреть

- Подозрительно длинные функции и файлы
- Повторяющиеся куски логики и условий

Итеративное улучшение ИИ-кода

Цикл работы

1. Генерация черновика ИИ
2. Анализ и поиск проблемных мест
3. Точечные запросы на рефакторинг
4. Повтор на шаг 2

Автоматизация: ИИ как анализатор запахов

Что можно поручить ИИ

- Поиск дублирования и длинных методов
- Проверку на принципы SOLID и DRY
- Предложения по упрощению сложной логики

Источники информации и изображений

1. https://medium.com/highape-tech/messy-code-v-s-clean-code-in-mvc-context-9ad99079a4f8
2. https://link.springer.com/chapter/10.1007/978-3-031-12774-8_5
3. https://www.freecodecamp.org/news/clean-coding-for-beginners/
4. https://habr.com/ru/news/789628/

Программная инженерия

Лекция 4: Качество кода. Чистый код и рефакторинг

Денисов Артём Владимирович

Уральский Федеральный Университет

2026